

Title

NoSQL Database Introduction – MongoDB Aggregation and Indexing

Objectives

1. To understand NoSQL database concepts.
 2. To work with MongoDB for document-based data storage.
 3. To perform CRUD (Create, Read, Update, Delete) operations.
 4. To use aggregation framework for statistical analysis.
 5. To create indexes and analyze performance improvements.
-

Problem Statement

Design and implement a **User Data Management System** using MongoDB that:

- Creates a collection for storing user data.
 - Performs CRUD operations.
 - Uses aggregation queries to calculate statistics (e.g., total sales, average age).
 - Creates indexes and observes query performance improvements.
-

Outcomes

After completing this experiment, students will be able to:

- Work with NoSQL document databases.
 - Perform CRUD operations in MongoDB.
 - Use aggregation pipeline for data analysis.
 - Implement indexing to optimize query performance.
 - Compare indexed vs non-indexed query execution.
-

Software and Hardware Requirements

Software

- MongoDB
- MongoDB Compass / Mongo Shell
- Node.js (optional for application integration)

Hardware

- Minimum 4GB RAM

- 20GB free disk space
 - Windows/Linux/Mac OS
-

Theory Concept (Brief)

◆ NoSQL Database

NoSQL databases store data in non-tabular formats such as documents, key-value pairs, graphs, or wide-column stores.

◆ MongoDB

MongoDB is a document-oriented NoSQL database that stores data in **BSON (Binary JSON)** format.

Key Features:

- Flexible schema
 - High scalability
 - Horizontal scaling
 - Aggregation framework
 - Indexing support
-

Database Design

Collection: users

Document Structure

```
{
  _id: ObjectId(),
  name: "Rahul",
  age: 25,
  department: "IT",
  salary: 50000,
  total_sales: 100000
}
```

Implementation

1 Create Collection

use SalesDB

```
db.createCollection("users")
```

2 CRUD Operations**

◆ Insert (Create)

```
db.users.insertMany([
  {name:"Rahul", age:25, department:"IT", salary:50000, total_sales:100000},
  {name:"Sneha", age:30, department:"HR", salary:40000, total_sales:50000},
  {name:"Amit", age:28, department:"IT", salary:60000, total_sales:150000}
])
```

◆ Read

```
db.users.find()
db.users.find({ department:"IT" })
```

◆ Update

```
db.users.updateOne(
  {name:"Rahul"},
  {$set:{salary:55000}}
)
```

◆ Delete

```
db.users.deleteOne({name:"Sneha"})
```

Aggregation Queries

1 Calculate Total Sales

```
db.users.aggregate([
  {
    $group: {
      _id: null,
      TotalSales: {$sum: "$total_sales"}
    }
  }
])
```

Output:

TotalSales

250000

2 Calculate Average Age

```
db.users.aggregate([
  {
    $group: {
      _id: null,
      AverageAge: {$avg: "$age"}
    }
  }
])
```

Output:

AverageAge

26.5

3 Department-wise Total Sales

```
db.users.aggregate([
  {
    $group: {
      _id: "$department",
      DepartmentSales: {$sum: "$total_sales"}
    }
  }
])
```

Indexing

◆ Create Index on Department

```
db.users.createIndex({ department:1 })
```

◆ Create Index on Age

```
db.users.createIndex({ age:1 })
```

◆ Check Indexes

```
db.users.getIndexes()
```

◆ Performance Observation

Without index:

```
db.users.find({ department:"IT" }).explain("executionStats")
```

- COLLSCAN (Collection Scan)
- Higher execution time

With index:

- IXSCAN (Index Scan)
 - Faster query execution
 - Reduced documents examined
-

Algorithm

CRUD Operations

1. Start
2. Create database and collection
3. Insert user documents
4. Retrieve records
5. Update records
6. Delete records
7. End

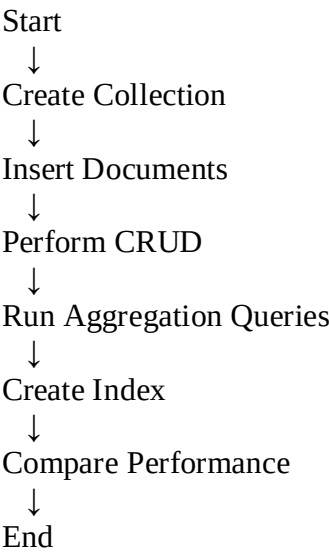
Aggregation

1. Start
2. Use \$group stage
3. Apply \$sum or \$avg
4. Display result
5. End

Indexing

1. Start
 2. Create index on frequently queried fields
 3. Run query with explain()
 4. Compare execution statistics
 5. End
-

Flowchart



Test Data Set

Name	Age	Department	Salary	Total_Sales
Rahul	25	IT	50000	100000
Sneha	30	HR	40000	50000
Amit	28	IT	60000	150000

Test Cases

Test Case	Operation	Expected Result	Status
TC1	Insert document	Document added	Pass
TC2	Update salary	Salary updated	Pass
TC3	Delete document	Record removed	Pass
TC4	Aggregate total sales	Correct total returned	Pass
TC5	Query with index	Faster execution	Pass

Conclusion

The MongoDB-based NoSQL system was successfully implemented. CRUD operations were performed efficiently. Aggregation queries provided statistical insights such as total sales and average age. Indexes significantly improved query performance by reducing collection scans. This experiment demonstrates the power of MongoDB in handling large-scale flexible data with efficient analytics capabilities.